Module 8: Natural Language Processing

Ultra-Comprehensive Guide to Building Intelligent Language Systems

Introduction

Natural Language Processing (NLP) enables machines to understand, interpret, and generate human language. It powers Google Translate, ChatGPT, virtual assistants, sentiment analysis, and countless applications that impact how humans communicate with technology.

The journey from basic text processing to modern transformer-based language models is one of the most fascinating evolutions in AI. This module takes you from understanding how to extract meaning from raw text, through word embeddings that capture semantic relationships, to building sophisticated sequence models like LSTMs and modern transformers.

By the end of this module, you'll understand the complete NLP pipeline and be able to build systems for machine translation, sentiment analysis, chatbots, and beyond.

PART 1: INTRODUCTION TO STATISTICAL NLP TECHNIQUES

What is NLP?

Natural Language Processing = Intersection of:

- Linguistics: Understanding language structure and meaning
- Computer Science: Algorithms and data structures
- Machine Learning: Learning patterns from data

Tasks:

- Text classification (spam detection, sentiment)
- Named Entity Recognition (find people, places, organizations)
- Machine translation (English → Spanish)
- Question answering (search engines)
- Summarization (extract key information)
- Dialogue (chatbots)

Text Preprocessing

Tokenization

Goal: Split text into tokens (words, subwords, characters)

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

text = "Hello! How are you? I'm fine."

# Sentence tokenization
sentences = sent_tokenize(text)
# Output: ["Hello!", "How are you?", "I'm fine."]

# Word tokenization
tokens = word_tokenize(text)
# Output: ["Hello", "!", "How", "are", "you", "?", "I", "'m", "fine", "."]

# Simple split (naive approach)
words = text.split()
# Output: ["Hello!", "How", "are", "you?", "I'm", "fine."]
# Problems: Punctuation attached, contractions not split
```

Stop Words

Definition: Common words with little semantic value

```
English stop words: the, a, an, and, or, is, was, are, in, on, at, ...

Reason for removal:
- Don't add meaning to classification
- Reduce noise
- Speed up processing
```

```
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))

tokens = ["Hello", "there", "world"]
filtered = [w for w in tokens if w.lower() not in stop_words]
# After removing "there": ["Hello", "world"]
```

When NOT to remove stop words:

- Machine translation (need complete meaning)
- Sentiment analysis ("not good" vs "good")
- LSTMs (capture all context)

Normalization

Goal: Reduce variations to canonical forms

```
Variations of same word:
- "running", "runs", "ran"  → All become "run"
- "HELLO", "Hello", "hello" → All become "hello"
```

Stemming

Idea: Remove affixes using rule-based approach

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = ["running", "runs", "runner", "ran"]

for word in words:
    print(f"{word} → {stemmer.stem(word)}")

# Output:
# running → run
# runs → run
# runner → runner  (doesn't remove -er reliably)
# ran → ran         (irregular, hard to handle)
```

Porter Stemmer Algorithm (simplified):

```
1. If word ends in "ies", replace with "i": abilities → abilitI → abil
2. If word ends in "ed", remove: produced → produc
3. If word ends in "ing" preceded by consonant, remove: running → run
... (many rules)
```

Problems:

- Over-stemming: "universal" → "univers"
- Under-stemming: "ran" not reduced to "run"
- Language-specific rules

Lemmatization

Idea: Convert to base form using linguistic knowledge

```
from nltk.stem import WordNetLemmatizer
import nltk

nltk.download('wordnet')
lemmatizer = WordNetLemmatizer()
```

```
words = ["running", "runs", "runner", "ran"]

for word in words:
    print(f"{word} → {lemmatizer.lemmatize(word)}")

# Output:
# running → running  (needs POS tag to work)
# runs → run         (works!)
# runner → runner    (needs training)
# ran → ran          (doesn't know past tense)

# With POS tags:
print(lemmatizer.lemmatize("running", pos="v"))  # running → run
print(lemmatizer.lemmatize("better", pos="a"))   # better → good
```

Comparison:

- **Stemming**: Fast, simple, sometimes inaccurate (Porter Stemmer)
- **Lemmatization**: Accurate, needs linguistic resources (WordNet)

Feature Extraction: From Text to Numbers

Bag of Words (BoW)

Idea: Represent text as count of word occurrences

```
Corpus:
  Doc1: "good movie"
  Doc2: "bad movie"
  Doc3: "good actor"

Vocabulary: {good, movie, bad, actor}

Representation:
  Doc1: [1, 1, 0, 0]  (1 "good", 1 "movie", 0 "bad", 0 "actor")
  Doc2: [0, 1, 1, 0]  (0 "good", 1 "movie", 1 "bad", 0 "actor")
  Doc3: [1, 0, 0, 1]
```

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "good movie",
    "bad movie",
    "good actor"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print(vectorizer.get_feature_names_out())  # ['actor' 'bad' 'good' 'movie']
```

```
print(X.toarray())
# [[0 0 1 1]
#  [0 1 0 1]
#  [1 0 1 0]]
```

Advantages:

- Simple, fast
- Works well for many tasks

Disadvantages:

- Loses word order ("good bad" vs "bad good")
- "the" same importance as "good"
- Doesn't capture semantics

TF-IDF (Term Frequency-Inverse Document Frequency)

Idea: Weight words by importance

TF (Term Frequency): How often word appears in document

$$TF(t, d) = \frac{\text{count of term } t \text{ in doc } d}{\text{total terms in doc } d}$$

IDF (Inverse Document Frequency): How rare word is across corpus

$$IDF(t) = \log\left(\frac{\text{total documents}}{\text{documents containing } t}\right)$$

TF-IDF:

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Intuition: Common words get low IDF (appear everywhere), rare discriminative words get high IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "good movie very good",
    "bad movie",
    "good actor good"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

print(vectorizer.get_feature_names_out())  # ['actor' 'bad' 'good' 'movie']
print(X.toarray())

# "good" has high TF in doc1 and doc3 (frequent)
```

```
# "bad" has low IDF (less discriminative)
# "actor" has high IDF (appears in few docs, discriminative)
```

TF-IDF vs Bag of Words:

Aspect	BoW	TF-IDF
Weight	Count	Importance
Common words	High weight	Low weight
Rare words	Same weight	Higher weight
Performance	Baseline	Usually better
Sparsity	Less sparse	More sparse

Language Modeling: Predicting Next Word

N-gram Model

Idea: Predict word based on previous N-1 words

Bigram (N=2):

```
"The cat sat on the mat"

P(word | previous) using counts:
- P("cat" | "The") = count("The cat") / count("The")
- P("sat" | "cat") = count("cat sat") / count("cat")
- P("on" | "sat") = count("sat on") / count("sat")
```

```
from nltk import bigrams, trigrams
from collections import Counter

text = "the cat sat on the mat".split()

# Get bigrams
bigram_list = list(bigrams(text))
# [('the', 'cat'), ('cat', 'sat'), ('sat', 'on'), ('on', 'the'), ('the', 'mat')]

# Count frequencies
bigram_freq = Counter(bigram_list)
print(bigram_freq)  # Counter({('the', ...): 1, ...})

# Calculate probability
def prob_next_word(word, bigram_freq):
    matching = [freq for (w1, w2), freq in bigram_freq.items() if w1 == word]
    return sum(matching) / sum(bigram_freq.values()) if matching else 0

# Probability "cat" follows "the"
print(prob_next_word("the", bigram_freq))
```

Limitations:

- Doesn't capture long-range dependencies
- Sparsity (many n-grams never seen)
- No semantic understanding

PART 2: WORD EMBEDDINGS

What are Word Embeddings?

Traditional Representation (One-hot encoding):

```
Vocabulary: {good, bad, movie, actor}
"good" → [1, 0, 0, 0]
"bad"  → [0, 1, 0, 0]

Problems:
- High-dimensional, sparse
- "good" and "bad" equally different (distance = √2)
- No semantic information
```

Word Embeddings (Dense vectors):

```
"good" → [0.2, 0.8, -0.3, 0.1]
"bad"  → [0.3, 0.1, -0.8, 0.2]

Benefits:
- Low-dimensional (50-300D), dense
- Similar words close (distance = 0.3)
- Captures semantics through relationships
```

Word2Vec: Skip-gram Model

Architecture

Goal: Given word, predict surrounding context words

```
Sentence: "the cat sat on the mat"
Word: "cat"
Context (window size 2): ["the", "sat"]

Network learns:
  Input: "cat" (one-hot)
  → Hidden layer (embedding)
  → Output: Probability over vocabulary for each context word

Loss: Minimize cross-entropy between predicted and actual context words
```

```
After training, hidden layer weights = word embeddings!
```

Skip-gram Process

Step 1: One-hot encode target word

```
"cat" → [0, 1, 0, 0, ...] (1 out of 10,000 vocab size)
```

Step 2: Multiply by embedding matrix

```
Embedding matrix W: (vocab_size, embedding_dim) = (10000, 100)
Multiply: [0, 1, 0, ...] × W = [w_23, w_24, ..., w_99]
Result: 100D embedding for "cat"
```

Step 3: Pass through output layer

```
Hidden layer embedding: [0.5, -0.2, 0.8, ..., 0.1]
Output layer W': (embedding_dim, vocab_size) = (100, 10000)
Output: [0.01, 0.5, 0.02, ..., 0.3] (probabilities over vocab)
```

Step 4: Predict context words

```
For context word "sat":
  Loss = -log(P(sat | cat))
  Update embeddings to maximize this probability
```

Word2Vec Implementation

```
from gensim.models import Word2Vec

# Training sentences
sentences = [
    "the cat sat on the mat".split(),
    "the dog sat on the log".split(),
    "a cat and a dog".split(),
]

# Train Word2Vec
model = Word2Vec(
    sentences=sentences,
    vector_size=100,      # Embedding dimension
    window=2,             # Context window size
    min_count=1,          # Minimum frequency
    sg=1,                 # Skip-gram model
    epochs=100
)

# Get word embedding
```

```
cat_embedding = model.wv['cat']  # 100D vector
print(cat_embedding.shape)  # (100,)

# Find similar words
print(model.wv.most_similar('cat', topn=5))
# [('dog', 0.89), ('animal', 0.85), ...]

# Word analogies
print(model.wv.most_similar(positive=['king', 'woman'], negative=['man']))
# ['queen', ...]  (king - man + woman ≈ queen)
```

Skip-gram vs CBOW

Skip-gram: Predict context from target

```
Target word "cat" → Predict ["the", "sat"]
Good for: Smaller datasets, good for rare words
```

CBOW (Continuous Bag of Words): Predict target from context

```
Context ["the", "sat"] → Predict "cat"
Good for: Larger datasets, faster training
```

GloVe: Global Vectors for Word Representation

Co-occurrence Matrix

Idea: Build global word co-occurrence statistics

```
Corpus: "cat sat mat", "dog sat log"

Co-occurrence matrix X:
       cat  sat  mat  dog  log
cat  [  0    1    1    0    0  ]  (cat appears with sat, mat)
sat  [  1    0    1    1    1  ]  (sat appears with cat, mat, dog, log)
mat  [  1    1    0    0    0  ]
dog  [  0    1    0    0    1  ]  (dog appears with sat, log)
log  [  0    1    0    1    0  ]

X[i,j] = number of times word i appears in context of word j
```

GloVe Optimization

Objective: Find embeddings that preserve co-occurrence probabilities

```
For each pair (i, j):
  Minimize: (w_i · w_j + b_i + b_j - log X[i,j])²
```

```
Where:
  w_i = embedding for word i
  b_i = bias term
  X[i,j] = co-occurrence count

Intuition: Dot product w_i · w_j should approximate log probability
```

GloVe Implementation

```
from gensim.models import KeyedVectors

# Load pre-trained GloVe embeddings
glove_model = KeyedVectors.load_word2vec_format('glove.6B.100d.txt', no_header=True)

# Get similarity
sim = glove_model.similarity('good', 'bad')
print(f"Similarity(good, bad): {sim:.3f}")  # ~0.2

sim = glove_model.similarity('good', 'great')
print(f"Similarity(good, great): {sim:.3f}")  # ~0.8 (higher, more similar)
```

PART 3: SEQUENTIAL MODELS

Recurrent Neural Networks (RNNs)

The Recurrence Idea

Traditional Neural Network:

```
Input → Dense → Dense → Output
No memory of previous inputs
Each sample processed independently
```

RNN:

```
Input₁ → RNN cell → Output₁
         ↓ (hidden state h₁)
Input₂ → RNN cell → Output₂
         ↓ (hidden state h₂)
Input₃ → RNN cell → Output₃
```

The recurrence: Cell's output at step t depends on:

1. Input at step t
2. Hidden state from step t-1

```
h_t = tanh(W_h * h_{t-1} + W_x * x_t + b)
y_t = softmax(W_y * h_t + b_y)
```

RNN Forward Pass

```
import numpy as np

def rnn_forward(X, Wh, Wx, Wy, bh, by):
    """
    X: (sequence_length, input_dim)
    Wh: (hidden_dim, hidden_dim) - hidden to hidden
    Wx: (input_dim, hidden_dim) - input to hidden
    Wy: (hidden_dim, output_dim) - hidden to output
    """
    sequence_length, input_dim = X.shape
    hidden_dim = Wh.shape[^0]

    # Initialize hidden state
    h_prev = np.zeros((hidden_dim, 1))

    outputs = []
    hidden_states = [h_prev]

    for t in range(sequence_length):
        x_t = X[t].reshape(-1, 1)

        # Recurrent computation
        h_t = np.tanh(np.dot(Wh, h_prev) + np.dot(Wx, x_t) + bh)

        # Output computation
        y_t = np.dot(Wy, h_t) + by

        outputs.append(y_t)
        hidden_states.append(h_t)
        h_prev = h_t

    return outputs, hidden_states
```

Vanishing Gradient Problem in RNNs

Issue: Gradients multiply across time steps

```
Loss L depends on h_1:
∂L/∂h_1 = ∂L/∂h_T · ∂h_T/∂h_{T-1} · ... · ∂h_2/∂h_1
        = gradient_T · gradient_{T-1} · ... · gradient_1

If each gradient &lt; 1 (common with tanh, sigmoid):
gradient_1 ≈ 0.9 × 0.9 × ... × 0.9 (100 times) ≈ 0.0000...

Result: Early time steps don't learn!
```

LSTM: Long Short-Term Memory Networks

The Solution: Memory Cell

LSTM = RNN with gates controlling information flow

Three Gates:

1. **Forget Gate**: Decide what to forget

```
f_t = σ(W_f * [h_{t-1}, x_t] + b_f)
c_t = c_{t-1} * f_t + ...

Output: 0 = forget completely, 1 = remember completely
```

2. **Input Gate**: Decide what to add

```
i_t = σ(W_i * [h_{t-1}, x_t] + b_i)
c̃_t = tanh(W_c * [h_{t-1}, x_t] + b_c)
c_t = c_{t-1} * f_t + i_t * c̃_t

"How much of new info to add?"
```

3. **Output Gate**: Decide what to output

```
o_t = σ(W_o * [h_{t-1}, x_t] + b_o)
h_t = o_t * tanh(c_t)

"Which parts of cell state to output?"
```

Why LSTM Works

Constant Error Flow:

```
Gradient through cell state:
∂L/∂c_1 = ∂L/∂c_T · ∂c_T/∂c_{T-1} · ... · ∂c_2/∂c_1
        = ... · f_T · f_{T-1} · ... · f_1

If f_t ≈ 1 (forget gate = remember):
Gradient ≈ 1 · 1 · ... · 1 = 1 (doesn't vanish!)
```

No vanishing gradient through cell state!

LSTM Implementation

```
import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([
    keras.layers.LSTM(128, input_shape=(sequence_length, input_dim), return_sequences=Tru
```

```
    keras.layers.LSTM(64),  # Last LSTM, output only final hidden state
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(output_dim, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=32)

# Prediction
predictions = model.predict(X_test)
```

Key Parameters:

- `units`: Hidden state dimension (128, 256, 512 typical)
- `return_sequences`: Return sequence of hidden states (for stacking LSTMs)
- `activation`: Usually 'tanh' (LSTM default)

PART 4: NLP APPLICATIONS

Sentiment Analysis

Problem

Goal: Classify text as positive, negative, or neutral

```
Review: "This movie is absolutely amazing!"
Label: Positive

Review: "Terrible plot and bad acting"
Label: Negative

Review: "The movie was okay"
Label: Neutral
```

Solution: LSTM for Sentiment

```
from tensorflow import keras
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Prepare data
texts = ["great movie", "bad film", "amazing performance"]
labels = [1, 0, 1]  # 1 = positive, 0 = negative

# Tokenize
tokenizer = Tokenizer(num_words=1000)
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)
X = pad_sequences(sequences, maxlen=20)  # Pad to length 20
```

```
# Model
model = keras.Sequential([
    keras.layers.Embedding(1000, 100, input_length=20),  # Word embeddings
    keras.layers.LSTM(64, dropout=0.2),  # LSTM layer
    keras.layers.Dense(1, activation='sigmoid')  # Binary classification
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X, labels, epochs=10, batch_size=32)

# Prediction
test_review = "This is wonderful"
test_seq = tokenizer.texts_to_sequences([test_review])
test_X = pad_sequences(test_seq, maxlen=20)
prediction = model.predict(test_X)
print(f"Sentiment: {prediction[^0][^0]:.2f}")  # Close to 1 = Positive
```

Machine Translation

Sequence-to-Sequence Model

Goal: Translate English to French

```
Input:  "Hello world"
Output: "Bonjour monde"

Architecture:
  Encoder (English): "Hello" → h_1, "world" → h_2 (gets context)
  Decoder (French):  Starts with h_2, predicts "Bonjour" → "monde"
```

Encoder-Decoder Implementation

```
from tensorflow import keras
from tensorflow.keras.layers import Input, LSTM, Dense, Embedding

# Hyperparameters
latent_dim = 256
num_encoder_tokens = 5000
num_decoder_tokens = 5000
max_encoder_length = 50
max_decoder_length = 50

# Encoder
encoder_inputs = Input(shape=(max_encoder_length,))
encoder_embedding = Embedding(num_encoder_tokens, 256)(encoder_inputs)
encoder_lstm = LSTM(latent_dim, return_state=True)
encoder_outputs, state_h, state_c = encoder_lstm(encoder_embedding)

# Use encoder states as decoder initial state
decoder_inputs = Input(shape=(max_decoder_length,))
```

```
decoder_embedding = Embedding(num_decoder_tokens, 256)(decoder_inputs)
decoder_lstm = LSTM(latent_dim, return_sequences=True, return_state=True)
decoder_outputs, _, _ = decoder_lstm(decoder_embedding, initial_state=[state_h, state_c])

# Output layer
decoder_dense = Dense(num_decoder_tokens, activation='softmax')
decoder_outputs = decoder_dense(decoder_outputs)

# Model
model = keras.Model([encoder_inputs, decoder_inputs], decoder_outputs)
model.compile(optimizer='adam', loss='categorical_crossentropy')

# Training
model.fit([encoder_data, decoder_data], target_data, epochs=50, batch_size=64)
```

Named Entity Recognition (NER)

Problem

Goal: Identify and classify named entities

```
Text: "Apple CEO Tim Cook announced new iPhone in Cupertino"

Entities:
- Apple: ORG (Organization)
- Tim Cook: PERSON
- iPhone: PRODUCT
- Cupertino: LOC (Location)
```

Sequence Labeling with LSTM

```
from tensorflow import keras

# Each word gets a label
# Sequence labeling approach

model = keras.Sequential([
    keras.layers.Embedding(vocab_size, 100, input_length=max_length),
    keras.layers.Bidirectional(keras.layers.LSTM(64, return_sequences=True)),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(num_entity_types, activation='softmax')
])

# Each position outputs entity label
# Input shape: (batch_size, sequence_length)
# Output shape: (batch_size, sequence_length, num_entity_types)
```

Chatbots

Simple Rule-Based Chatbot

```
import random

# Pattern matching
patterns = {
    "hello": ["Hello! How can I help?", "Hi there!"],
    "bye": ["Goodbye!", "See you later!"],
    "how are you": ["I'm doing great!", "Pretty good, thanks for asking!"]
}

def chatbot_response(user_input):
    user_input = user_input.lower()

    for pattern, responses in patterns.items():
        if pattern in user_input:
            return random.choice(responses)

    return "I don't understand. Can you rephrase?"

# Usage
while True:
    user_input = input("You: ")
    response = chatbot_response(user_input)
    print(f"Bot: {response}")
```

Advanced: Seq2Seq Chatbot

```
# Similar to machine translation:
# User input (encoder) → Generate response (decoder)

# Training data: (user_input, chatbot_response) pairs
training_data = [
    ("Hello", "Hi there!"),
    ("How are you", "I'm doing great!"),
    ("What's your name", "I'm a chatbot"),
    ...
]

# Use encoder-decoder model (like machine translation)
# Encoder: Encode user message
# Decoder: Generate response

# For production, use pre-trained models (GPT-2, BERT-based chatbots)
```

Advanced Topics

Attention Mechanism

Problem: Encoder final state loses information

Solution: Attention allows decoder to "look at" all encoder states

```
Decoder at step t:
  1. Compute attention weights for each encoder output
  2. Weighted sum (context vector)
  3. Concatenate with decoder input
  4. Predict next word

Result: Decoder can focus on relevant parts of input!
```

Transformer Architecture

Revolution: Replace RNN with self-attention (2017)

```
Key advantages:
- Parallel processing (RNN sequential)
- Longer range dependencies
- Scales better to large datasets
- Foundation for BERT, GPT, modern LLMs
```

References

1. Bengio, Y., Ducharme, R., Vincent, P., & Janvin, C. (2023). A neural probabilistic language model. The journal of machine learning research, 3, 1137-1155.
2. Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2023). Distributed representations of words and phrases and their compositionality. In Advances in neural information processing systems (pp. 3111-3119).
3. Pennington, J., Socher, R., & Manning, C. D. (2023). Glove: Global vectors for word representation. In Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP) (Vol. 14, pp. 1532-1543).
4. Hochreiter, S., & Schmidhuber, J. (2023). Long short-term memory. Neural computation, 9(8), 1735-1780.
5. Sutskever, I., Vinyals, O., & Le, Q. V. (2023). Sequence to sequence learning with neural networks. In Advances in neural information processing systems (pp. 3104-3112).
6. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2023). Attention is all you need. In Advances in neural information processing systems (pp. 5998-6008).
7. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2023). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

8. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2023). Language models are unsupervised multitask learners. OpenAI blog, 1(8), 9.
9. NLTK Documentation. (2024). Natural Language Toolkit. Retrieved from https://www.nltk.org/
10. Gensim Documentation. (2024). Topic Modelling for Humans. Retrieved from https://radimrehurek.com/gensim/
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20]

⁂

1. https://www.ibm.com/think/topics/backpropagation
2. https://arxiv.org/html/2408.10006v1
3. https://nlp.stanford.edu/projects/glove/
4. https://towardsdatascience.com/word2vec-glove-and-fasttext-explained-215a5cd4c06f/
5. https://www.ibm.com/think/topics/word-embeddings
6. https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/
7. https://www.linkedin.com/pulse/mastering-word-embeddings-nlp-comprehensive-journey-through-sachin-imekc
8. https://www.geeksforgeeks.org/computer-vision/transfer-learning-for-computer-vision/
9. https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0300767
10. https://nlp2024.jeju.ai/en/week04/index.html
11. https://arxiv.org/abs/2406.10050
12. https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/
13. https://www.oncologyradiotherapy.com/articles/mcripunet-a-modified-unet-architecture-for-semantic--segmentation-of-images-for-breast-cancer-detection.pdf
14. https://towardsdatascience.com/neural-networks-backpropagation-by-dr-lihi-gur-arie-27be67d8fdce/
15. https://en.wikipedia.org/wiki/Backpropagation
16. https://pub.towardsai.net/backpropagation-2eeb25201095
17. https://viso.ai/deep-learning/convolution-operations/
18. https://www.sciencedirect.com/topics/computer-science/long-short-term-memory-network
19. https://www.linkedin.com/pulse/neural-network-chain-rule-understanding-algorithm-deep-doug-rose-maaie
20. https://ijcat.com/archieve/volume13/issue6/ijcatr13061003.pdf